CSE314 Bash Scripting Online - Section A2

May 18, 2025

Website Link Integrity Checker
Time: 45 minutes

Overview

You have joined the development team at TechDocs Inc. as a DevOps engineer. The company runs a large documentation portal with thousands of files that are updated by multiple teams. A common issue is broken internal links between documents, which frustrates users and decreases the portal's reliability.
Your task is to create a bash script that identifies broken internal links in a website directory, focusing on efficiency through a simple caching system. This tool will help both the automated build pipeline and developers who need to check their changes locally.

Objectives

1. Create a script that scans files for internal links (check only the ones with `<a href="...">`) and verify if they point to existing resources. (In simple words, check if the file exists or not)
2. Implement a basic caching system to avoid rechecking unchanged files
3. Generate a clear report of broken links and basic statistics

Requirements

1. File Scanning and Link Verification

The script must scan for HTML files in the provided directory and its subdirectories. Extract internal links from HTML files using pattern matching and verify if target files exist in the directory structure. A link is "broken" if its target file is missing.

2. Simple Caching System

Implement a cache file (name `cache.txt` storing file paths and last scanned timestamps. **Only check files that are new or modified since the last scan.** You can take help of the following commands -

- To get the last modification timestamp of a file, use this command
  `stat -format=%Y <file_name>`
- To get the current system timestamp use: `date +%s`

3. Reporting Output

Generate a report listing all files' checked summary. It will contain scan time, broken links for each HTML file, and some additional statistics. Sample output for report and cache file is shown in expected outputs.

Command

```
Usage: ./script.sh <directory> [cache_file_path]
```

If `cache_file_path` not provided, it will open a new `cache.txt` file and store the data specified above. Otherwise, use the provided file path as cache.

Sample Directory Structure

- **docs/**
  - getting-started/
    - index.html (links to: ../reference/api.html, installation.html)
    - installation.html (links to: index.html, ../tutorials/basic.html)
    - faq.html (links to: nonexistent.html)
  - tutorials/
    - index.html (links to: basic.html, advanced.html)
    - basic.html (links to: index.html, ../reference/api.html)
    - advanced.html (links to: basic.html, ../reference/config.html)
  - reference/
    - index.html (links to: api.html, config.html)
    - api.html (links to: index.html)
    - config.html (links to: api.html, index.html)

Expected Report Output

```
Scan time: 2025-05-18 14:32:45
Directory: /home/user/website/docs

BROKEN LINK COUNTS:
docs/getting-started/faq.html: 2
docs/already-started/faqv2.html: 4
docs/getting-wasted/faq23.html: 0

STATISTICS:
Total HTML files scanned: 9
Files checked: 3 (6 from cache)
Total internal links: 20
Broken links: 6

CACHE SUMMARY:
Cache hits: 6
Cache misses: 3
```

Cache File Example

```
Cache File - Last updated: 2025-05-18 14:32:45
docs/getting-started/index.html:1684445923
docs/getting-started/installation.html:1684445930
docs/getting-started/faq.html:1684445935
docs/tutorials/index.html:1684445940
docs/tutorials/basic.html:1684445945
docs/tutorials/advanced.html:1684445950
docs/reference/index.html:1684445955
docs/reference/api.html:1684445960
docs/reference/config.html:1684445965
```

Submission Guidelines

Important Notes

- Using the internet during the assignment is strictly prohibited
- Submit your solution in Moodle - failure to do so will result in mark deduction
- The filename should be **online-StudentID.sh** (e.g., `online-2105XXX.sh`)
- Make sure your script is executable